



LeetCode 231: Power of Two

1. Problem Title & Link

Title: LeetCode 231: Power of Two

Link: <https://leetcode.com/problems/power-of-two/>

2. Problem Statement (Short Summary)

Given an integer n , determine whether it is a power of 2.

Return:

- true if n can be written as 2^x
- false otherwise

Examples of powers of 2:

$$1 = 2^0$$

$$2 = 2^1$$

$$4 = 2^2$$

$$8 = 2^3$$

$$16 = 2^4$$

$$32 = 2^5$$

Examples that are NOT powers of 2:

3, 5, 6, 7, 10, 12

3. Examples (Input → Output)

Example 1

Input:

$$n = 1$$

Output:

true

Explanation:

$$1 = 2^0$$

Example 2

Input:

$$n = 16$$

Output:

true

Explanation:

$$16 = 2^4$$



Example 3

Input:

$n = 3$

Output:

false

Explanation:

3 cannot be expressed as 2^x

Example 4

Input:

$n = 0$

Output:

false

4. Constraints

$-2^{31} \leq n \leq 2^{31} - 1$

Important:

Negative numbers cannot be powers of two.

0 is not a power of two.

5. Core Concept (Pattern / Topic)

Bit Manipulation

Secondary Topics:

- Binary Representation
- Mathematics

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Keep dividing by 2.

If eventually we reach 1 $\rightarrow$ Power of Two.

If at some point division is not possible $\rightarrow$ Not Power of Two.

Example:

16

$16 / 2 = 8$



$$8 / 2 = 4$$

$$4 / 2 = 2$$

$$2 / 2 = 1$$

Reached 1

Answer = true

Better Observation

Let's write powers of two in binary.

$$1 = 0001$$

$$2 = 0010$$

$$4 = 0100$$

$$8 = 1000$$

$$16 = 10000$$

$$32 = 100000$$

Notice:

Power of Two

=

Exactly ONE bit is set

Now observe:

$$8 = 1000$$

$$8 - 1 = 0111$$

AND Operation:

$$1000$$

$$0111$$

$$0000$$

Result becomes zero.

Another example:

$$16 = 10000$$

$$15 = 01111$$

$$10000$$

$$01111$$

$$00000$$

Again zero.



But for non-power of two:

10 = 1010

9 = 1001

1010

1001

1000

Not zero.

Key Property

For every positive power of two:

$n \& (n - 1) = 0$

This is the fastest solution.

7. Visual / Intuition Diagram (ASCII Diagram)

Power of Two

16

Binary

10000

15

01111

AND

10000

01111

00000

Result:

0

Hence TRUE.

Not Power of Two

10



```
1010

9

1001

AND

1010
1001
----
1000
Result:
1000 ≠ 0
Hence FALSE.
```

8. Pseudocode (Language Independent)

```
if n <= 0
    return false

if (n AND (n-1)) == 0
    return true

return false
```

9. Code Implementation

Python

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:

        if n <= 0:
            return False

        return (n & (n - 1)) == 0
```

Java

```
class Solution {

    public boolean isPowerOfTwo(int n) {
```



```
    if(n <= 0)
        return false;

    return (n & (n - 1)) == 0;
}
```

10. Time & Space Complexity

Bit Manipulation Solution

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Why?

Only one AND operation is performed.

No loops.

No extra data structures.

11. Common Mistakes / Edge Cases

Mistake 1

Forgetting to check:

$n \leq 0$

Example:

$n = 0$

$0 \& (-1) = 0$

Without validation:

Wrong Answer = true

Mistake 2

Using floating-point logarithms.

Example:

$\log_2(1024)$

May suffer precision issues for very large numbers.

Edge Cases

Input	Output
1	TRUE



2	TRUE
0	FALSE
-8	FALSE
3	FALSE

12. Detailed Dry Run (Step-by-Step Table)

Input:

$n = 16$

Step 1

Binary representation:

$16 = 10000$

Step 2

Compute:

$n - 1 = 15$

Binary:

$15 = 01111$

Step 3

Apply AND

Value	Binary
16	10000
15	1111

10000

01111

00000

Step 4

Check result

0

Condition:

$(n \& (n-1)) == 0$

TRUE

Output:

true



13. Common Use Cases (Real-Life / Interview)

Memory Allocation

Many systems allocate memory blocks as:

2 KB

4 KB

8 KB

16 KB

32 KB

64 KB

Checking whether size is power of two is common.

Networking

Subnet calculations

Routing tables

Buffer sizes

Game Development

Texture sizes

128×128

256×256

512×512

1024×1024

Usually powers of two.

Competitive Programming

Used heavily in:

Bitmasking

Segment Trees

Fenwick Trees

Sparse Tables

14. Common Traps (Important!)

Trap 1

Using:

$n \% 2 == 0$

This only checks whether number is even.



Example:

6

Even but NOT power of two.

Trap 2

Ignoring negative numbers.

-8

Not a power of two.

Trap 3

Confusing:

Power of Two

with

Multiple of Two

Example:

12

Multiple of 2

But NOT power of 2.

15. Builds To (Related LeetCode Problems)

Learning progression:

LC 231 - Power of Two



LC 191 - Number of 1 Bits



LC 338 - Counting Bits



LC 190 - Reverse Bits



LC 201 - Bitwise AND of Numbers Range



LC 137 - Single Number II

These build strong Bit Manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
----------	------	-------	------------------



Repeated Division	$O(\log n)$	$O(1)$	Good
Logarithm	$O(1)$	$O(1)$	Risky
Bit Manipulation	$O(1)$	$O(1)$	Best

Repeated Division

while $n \% 2 == 0$:

$n /= 2$

Simple but slower.

Bit Manipulation

$(n \& (n - 1)) == 0$

Industry-standard solution.

17. Why This Solution Works (Short Intuition)

A power of two contains exactly one set bit in its binary representation.

Subtracting 1 flips that set bit and all bits after it.

Therefore:

$n \& (n - 1)$

always becomes:

0

only for powers of two.

18. Variations / Follow-Up Questions

Follow-Up 1

Check whether a number is a power of 4.

Related:

Cracking the Coding Interview style interview question.

Follow-Up 2

Count how many powers of two exist in an array.

Example:

[1,2,3,4,5,8]

Answer:

4

Follow-Up 3

Find the nearest power of two greater than a given number.



Example:

$n = 13$

Answer = 16

Follow-Up 4

Can you solve it recursively?

$n = n / 2$

until $n == 1$

Possible, but less efficient than bit manipulation.

Interview Takeaway:

Whenever you see **Power of Two**, immediately remember:

$n > 0 \ \&\& \ (n \ \& \ (n - 1)) == 0$

This is one of the most frequently asked Bit Manipulation tricks in coding interviews.